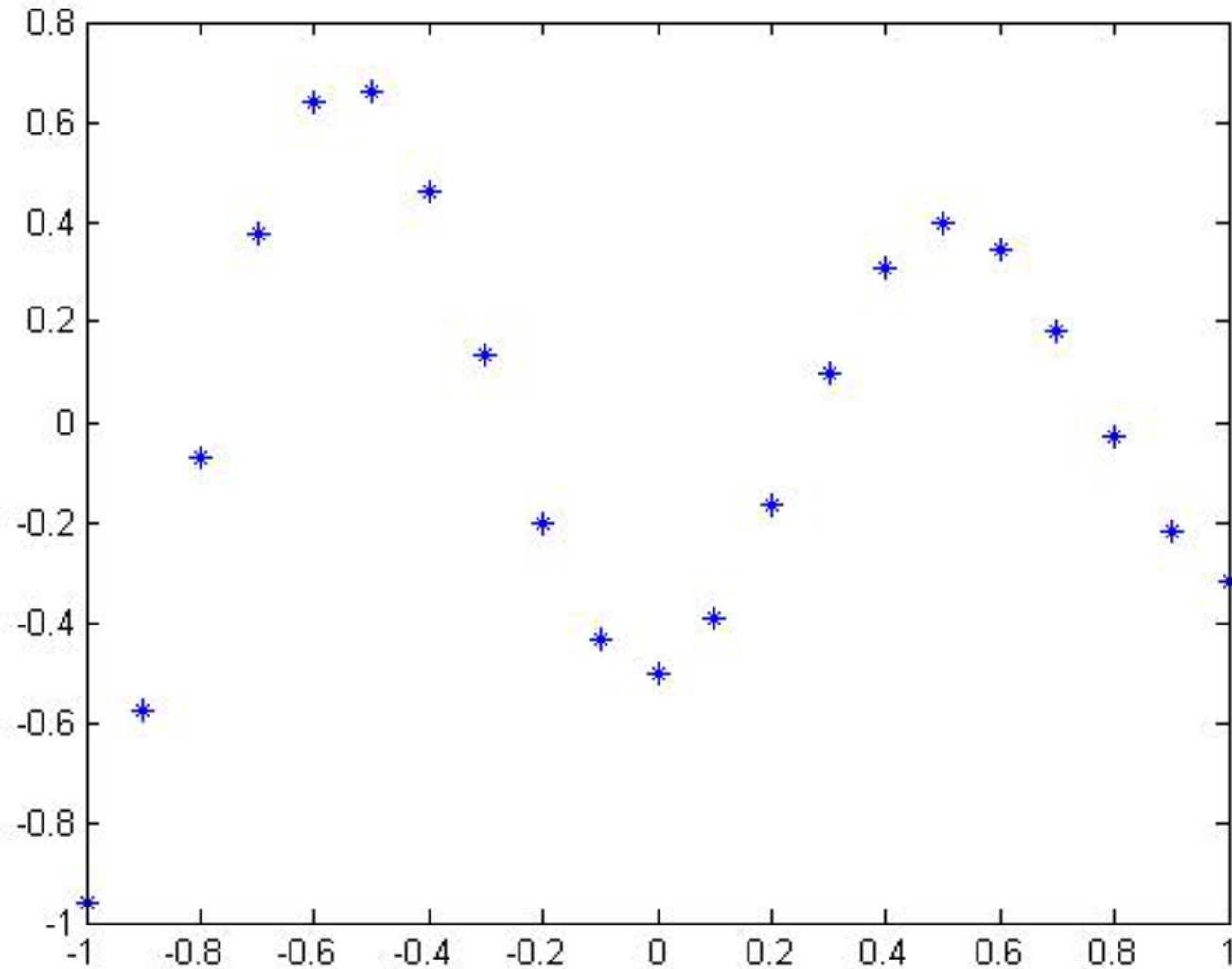


Deep Learning using Convolutional Neural Networks

1. Fundamentals
2. Motivation
3. Structure of CNN
4. CNN Regularization Methods

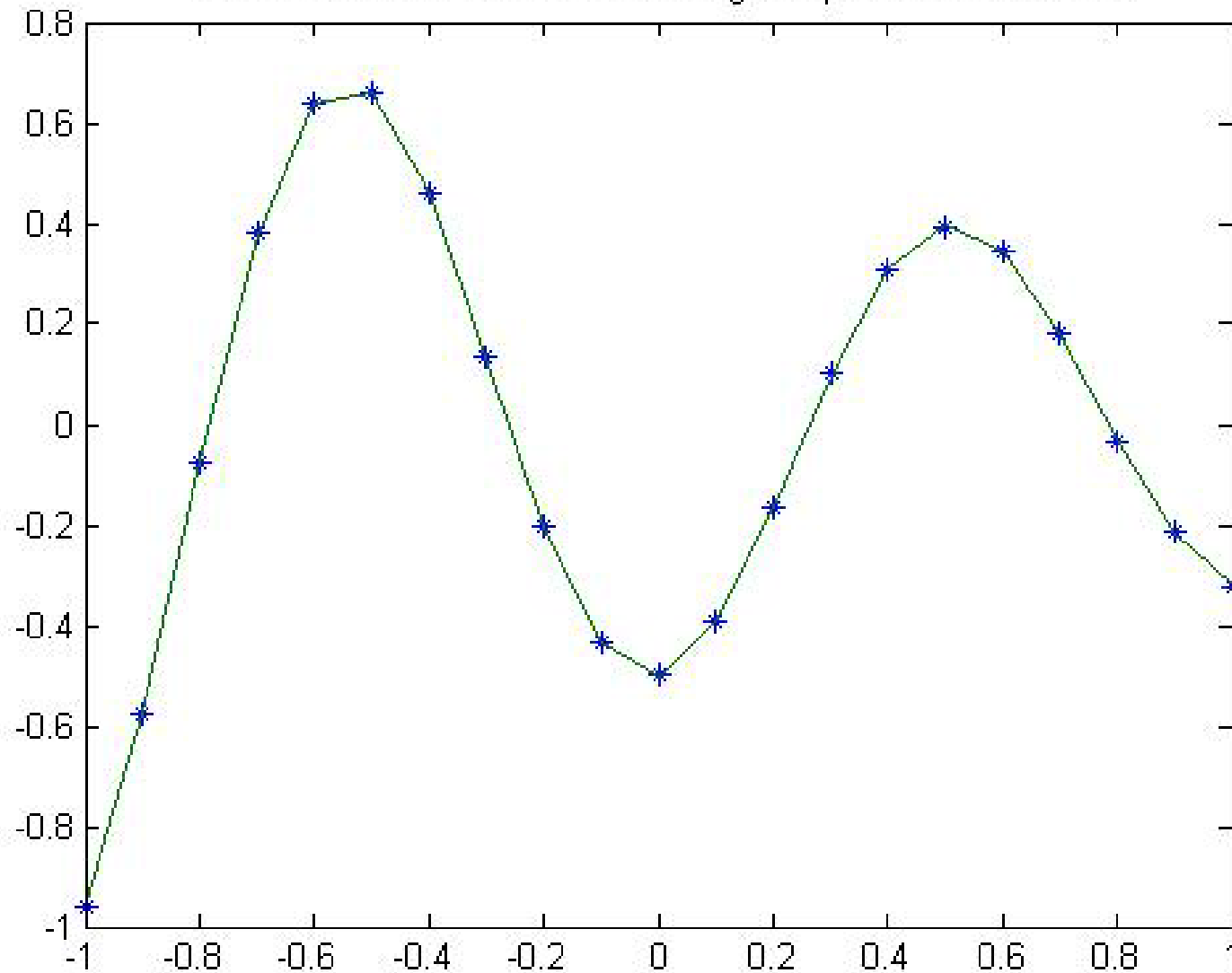
1. Overfitting Problem with neural networks



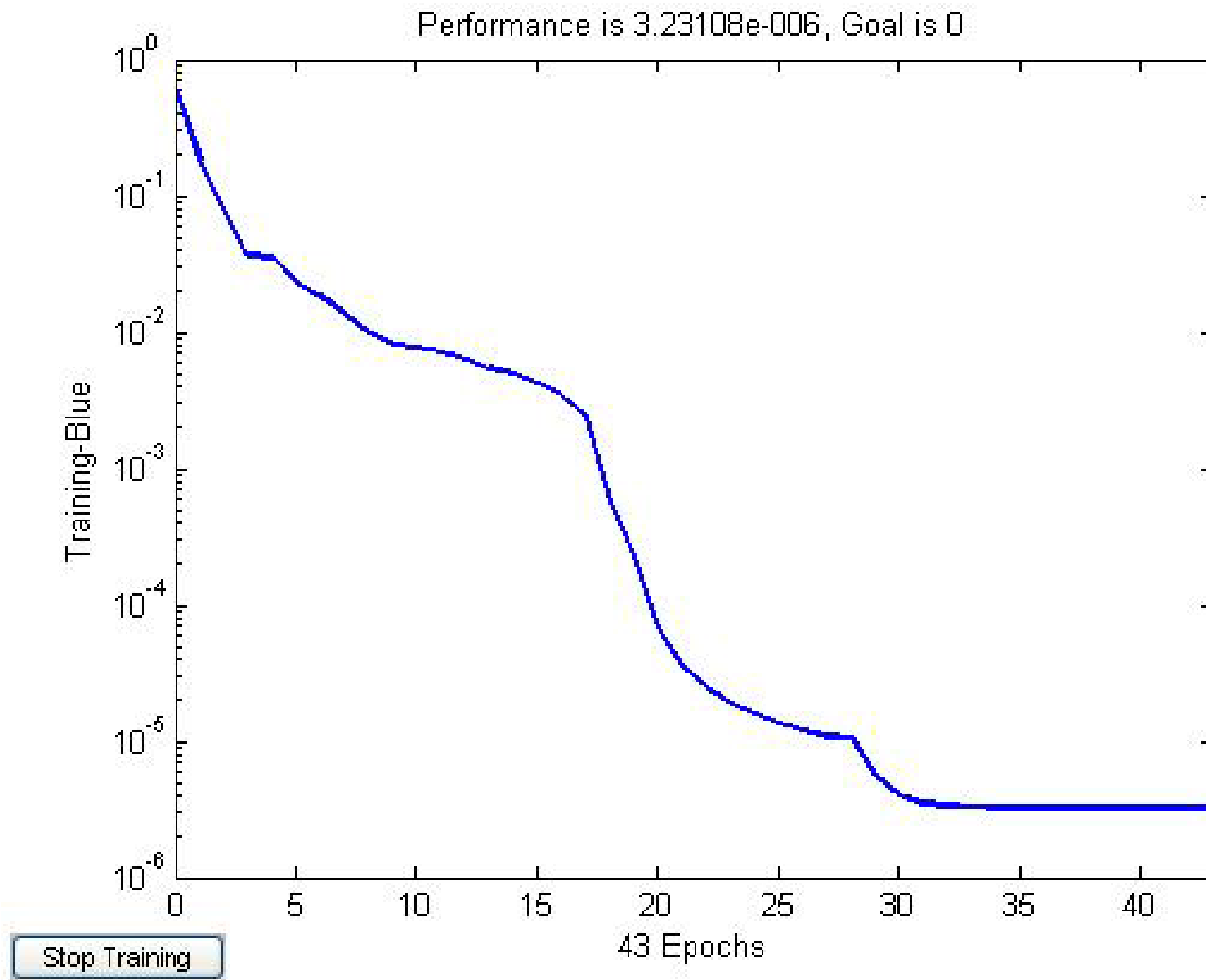
21 data points from an unknown function

5 hidden nodes: one hidden layer with sigmoid

Small difference between the training samples and NN results

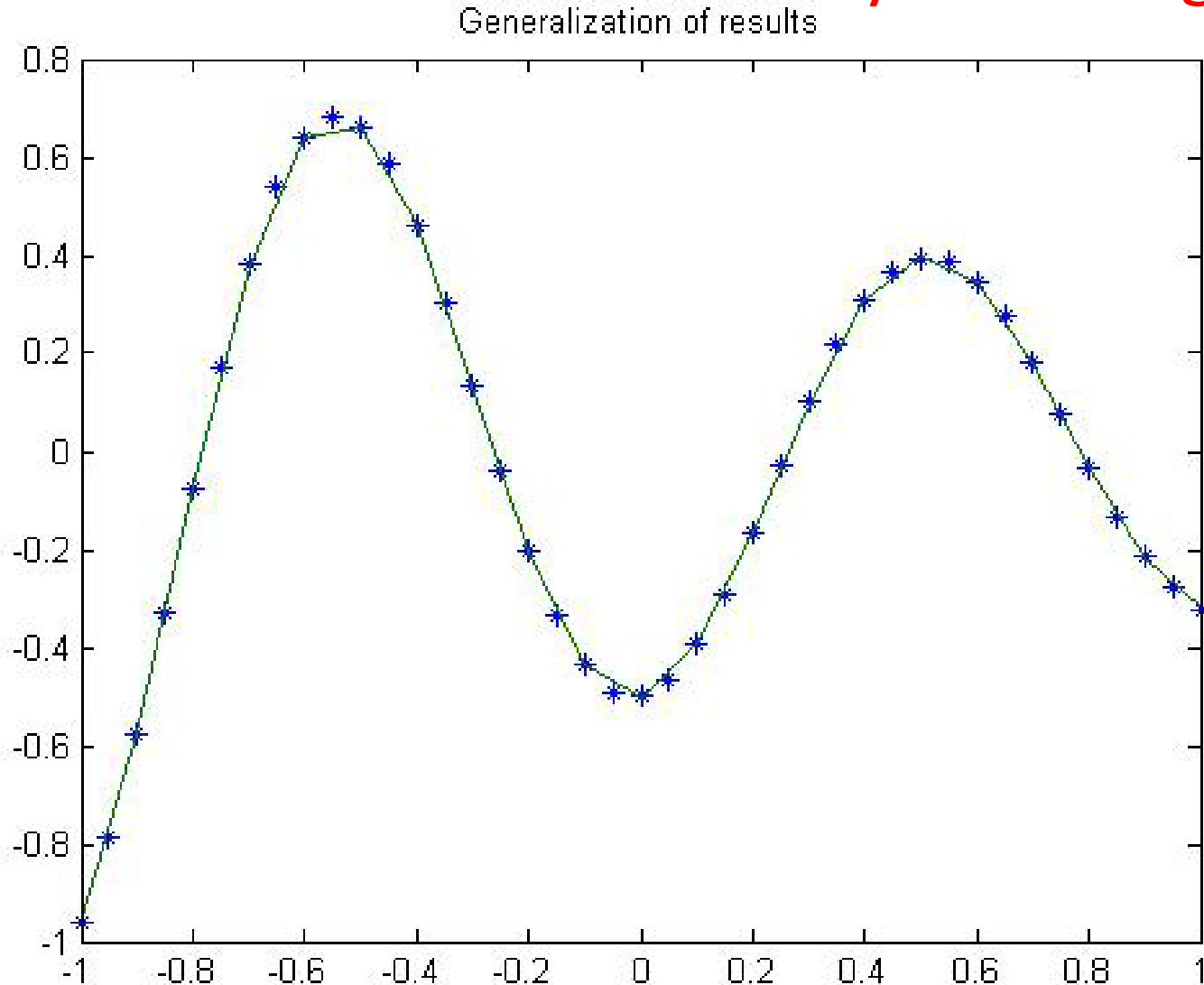


Result after training of the ANN



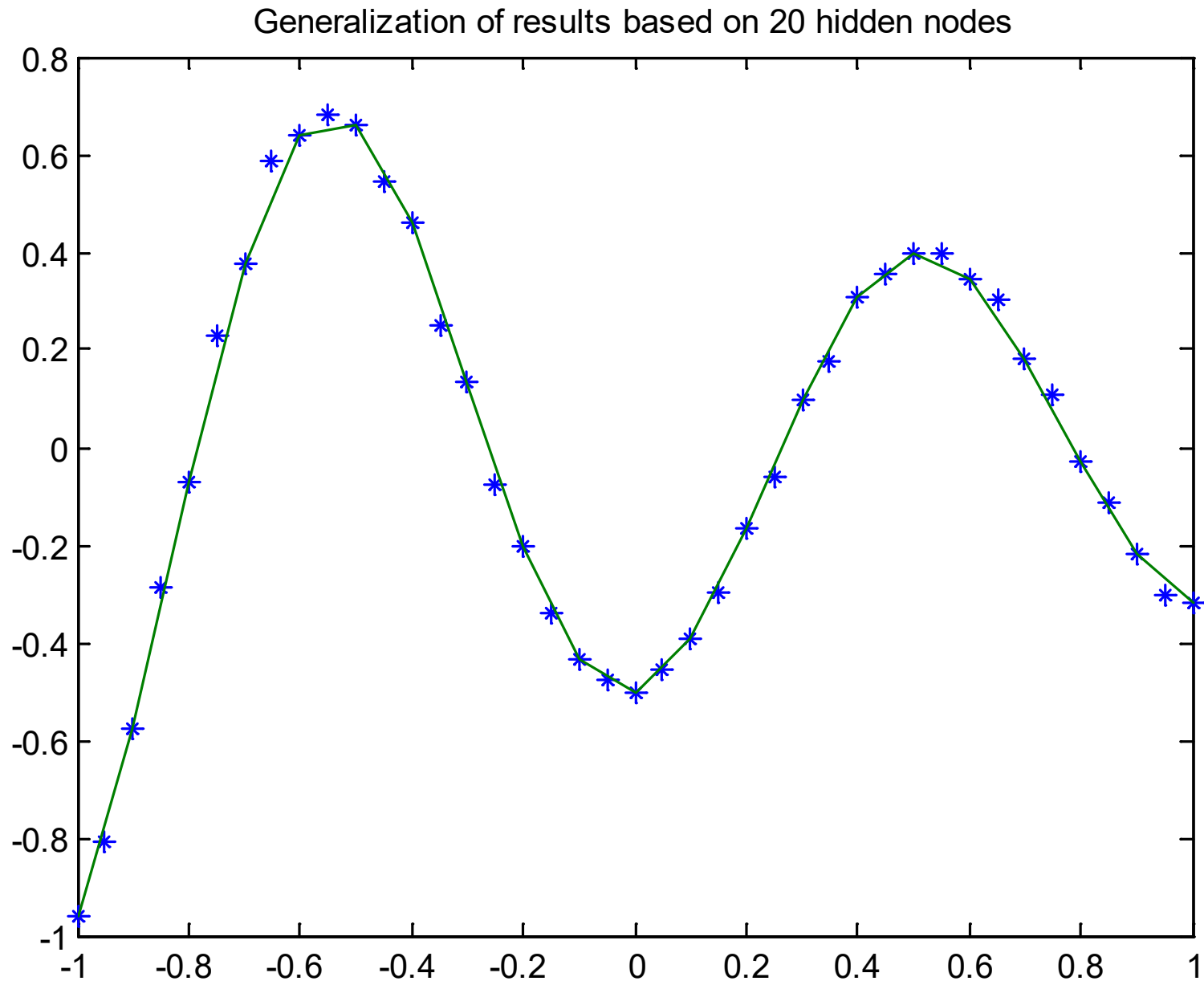
Error decreasing means convergence of the ANN

5 hidden nodes: one hidden layer with sigmoid

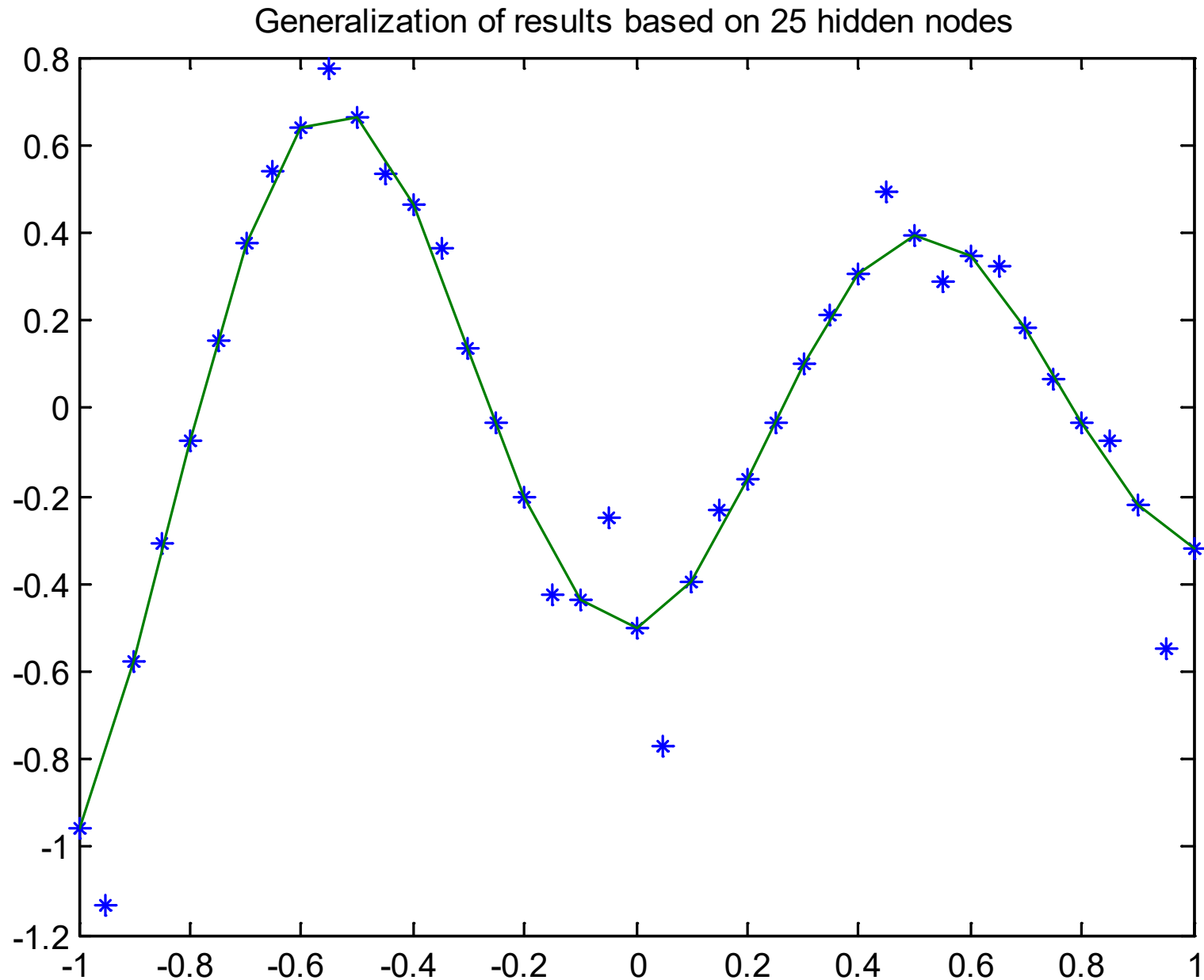


Demonstrate the generalization property of the ANN
The ANN gives reasonable results at -0.95, -0.85, -0.75 etc.

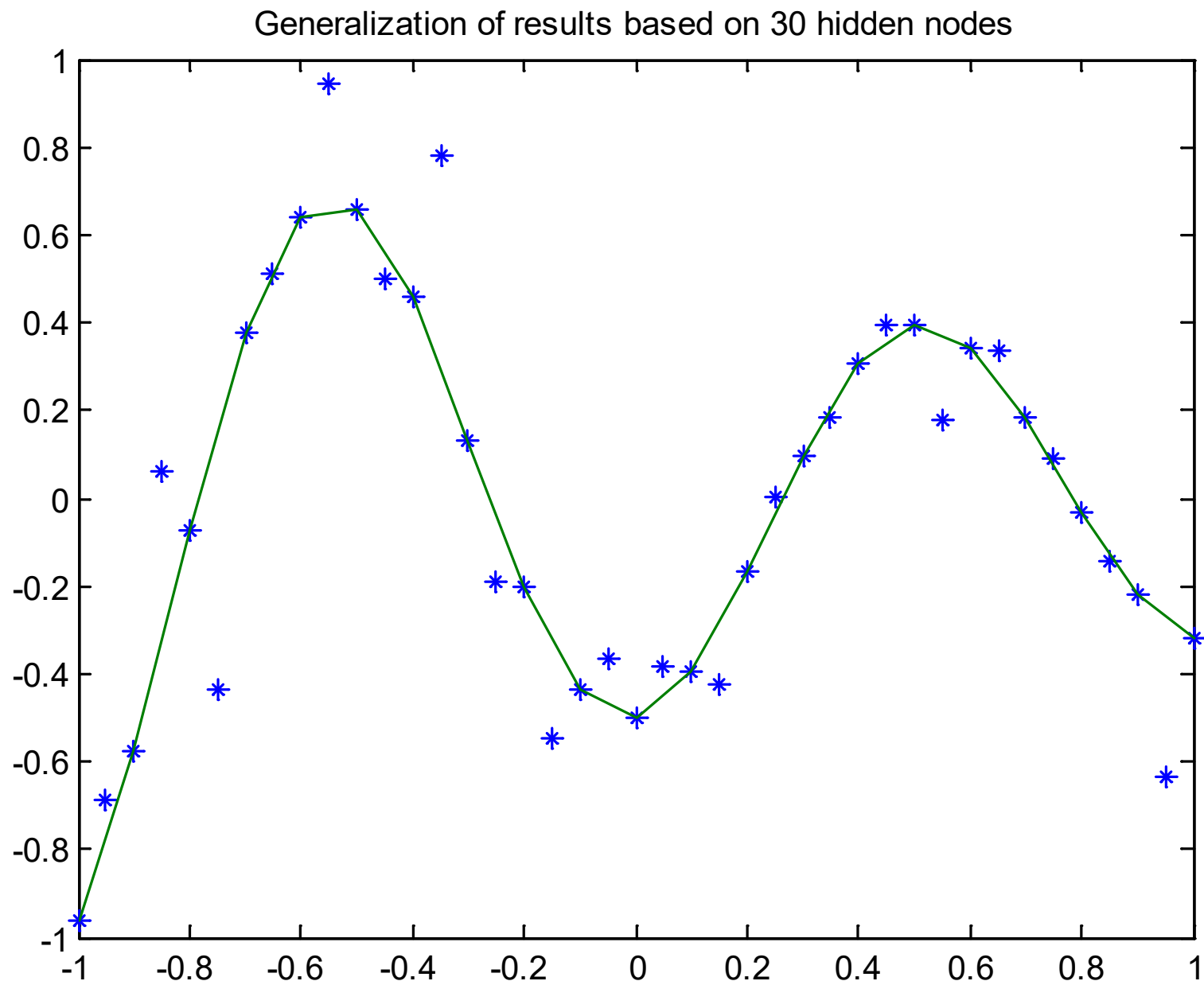
20 hidden nodes: one hidden layer with sigmoid



25 hidden nodes: one hidden layer with sigmoid

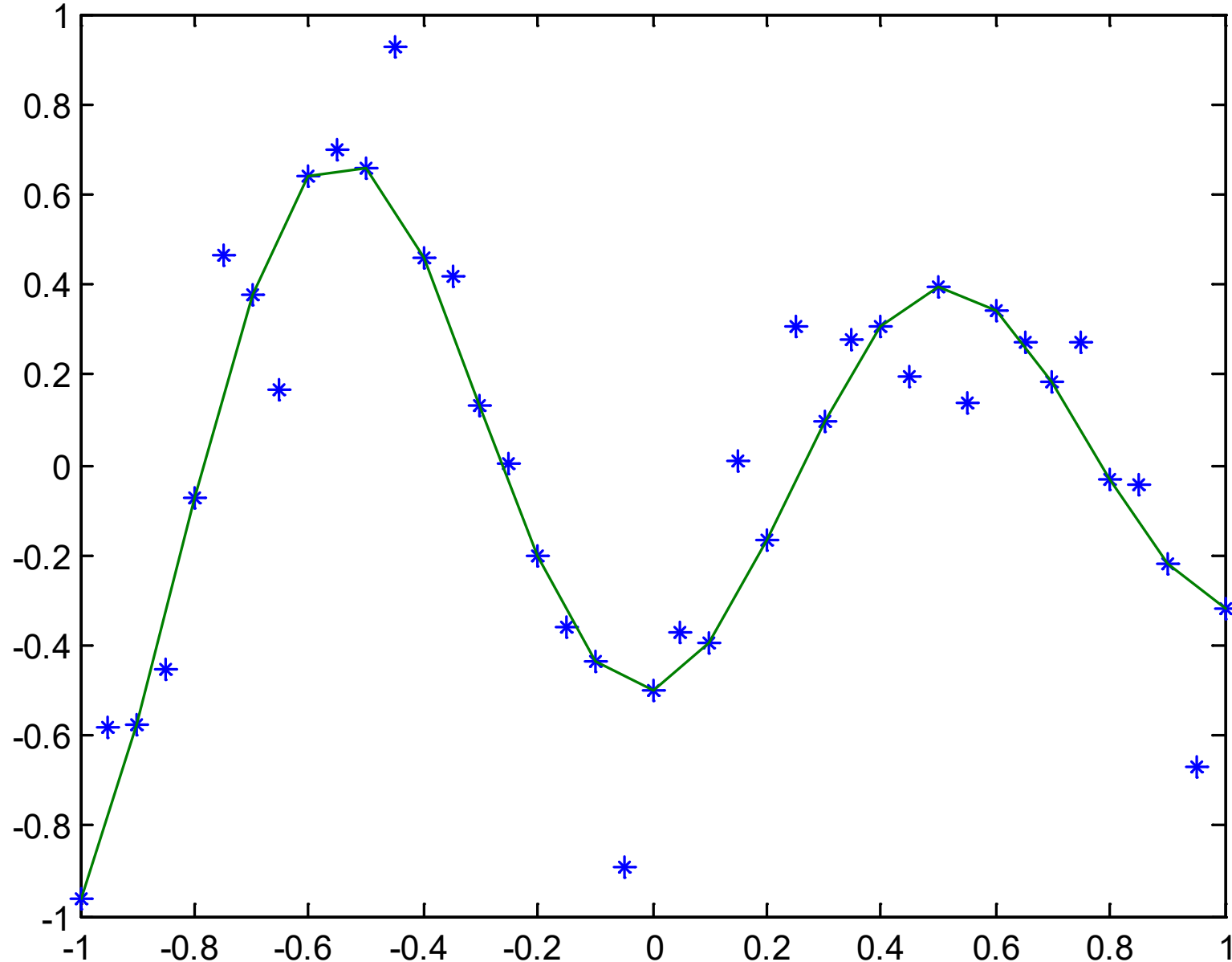


30 hidden nodes: one hidden layer with sigmoid



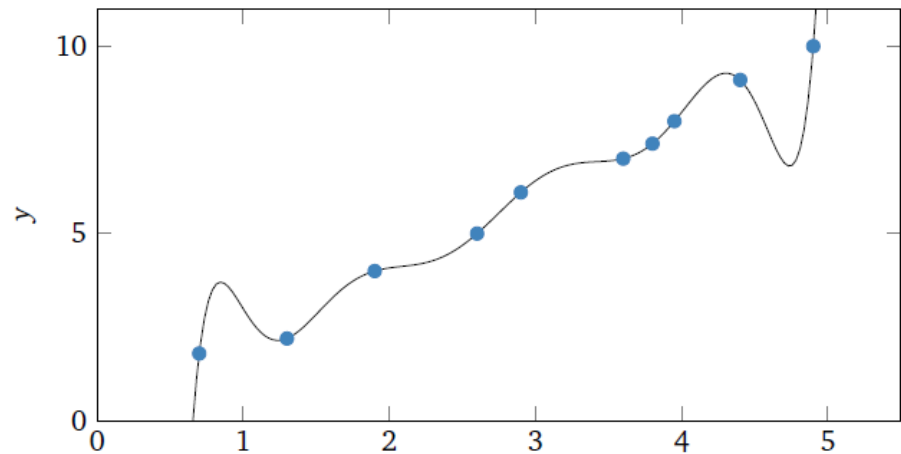
40 hidden nodes: one hidden layer with sigmoid

Generalization of results based on 40 hidden nodes



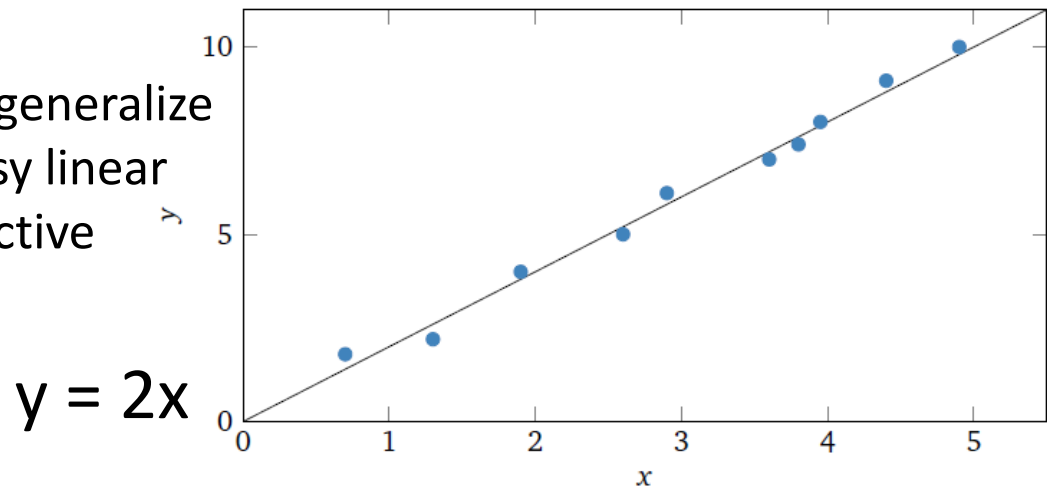
Regularization to deal with overfitting ...

with an aim to improve the accuracy of a deep learning model



9-th-order polynomial $y = a_0x^9 + a_1x^8 + \dots + a_9$ the linear model $y = 2x$:

The high order model fails to generalize to other data points. The noisy linear model will have greater predictive capability.

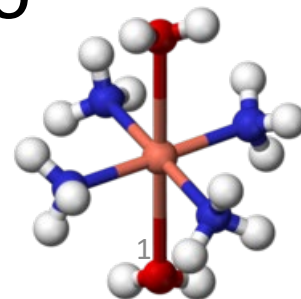


Overfitting is caused by noise in the training data that the neural network picks up during training. This learned noise, however, is unique to each training set. **As soon as the model sees new data from the same problem domain, but without that noise, the performance of the neural network gets much worse.**

Small weights: the network will not change too much when the inputs is changed with local noise.

Unregularized network uses large weights to learn a complex model that carries a lot of information about the noise in the training data.

Complex neural networks are susceptible to overfitting. To prevent overfitting, we use **regularization methods.**



Regularization in Deep Learning — L1, L2, Dropout & Early stopping, Batch normalization and Data augmentation.

Regularization is a set of techniques that can prevent overfitting in neural networks and thus improve the accuracy of a Deep Learning Model when facing completely new data from the problem domain.

Overfitting

Overfitting refers to the phenomenon where **a neural network models the training data very well but fails when it sees new data from the same problem domain.**

- Overfitting can manifest in various ways, such as high variance in the model's predictions or excessive sensitivity to small changes in the input data.
- Regularization techniques help strike a balance between fitting the training data well and maintaining the model's ability to generalize to new data.

“Why does the neural network picks up that noise in the first place?”

The reason for this is that the complexity of this network is too high. A fit of a neural network with higher complexity is shown in the image on the right-hand side.

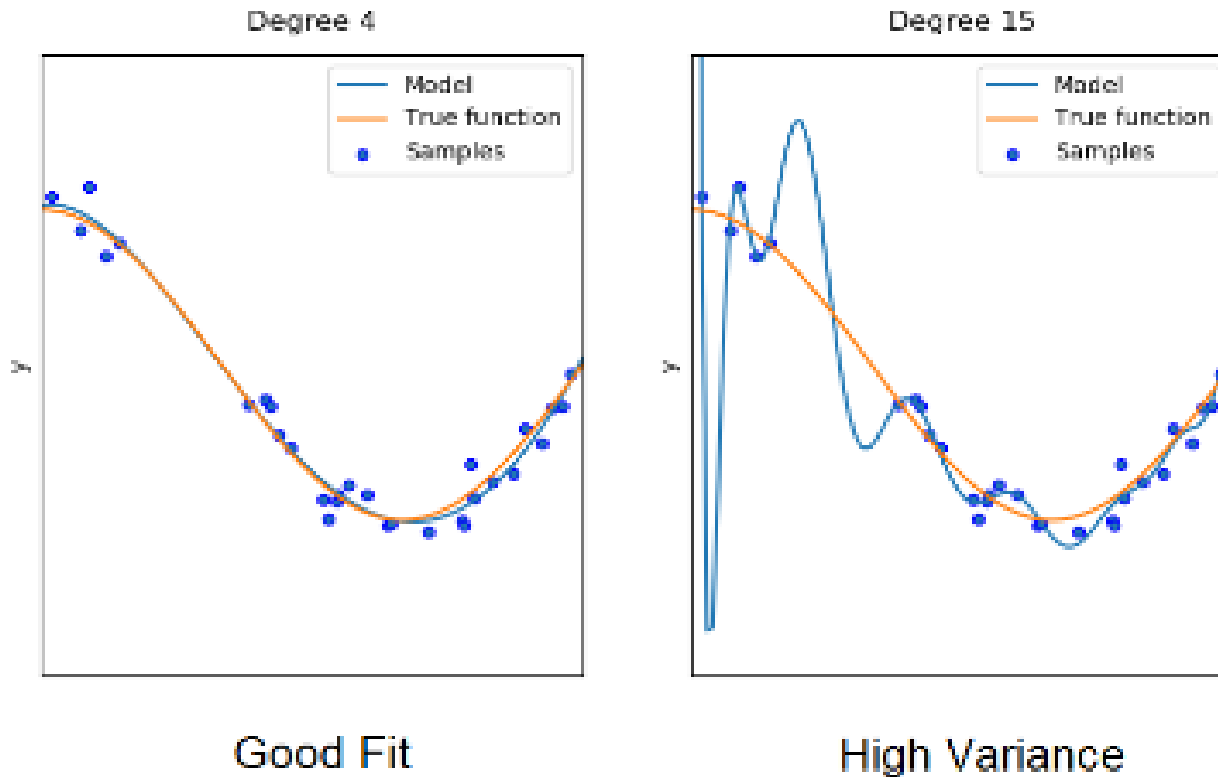


Figure. Model with a good fit and the other one with high variance.

Source: <https://www.researchgate.net/publication/332412613>

With reference to previous figure ...

- The model with a higher complexity is able to pick up and learn patterns (noise) in the data that are just caused by some random fluctuation or error. The network would be able to model each data sample of the distribution one-by-one, while not recognizing the true function that describes the distribution.
- New arbitrary samples generated with the true function would have a high distance to the fit of the model. We also say that the model has a high variance.
- On the other hand, **the lower complexity network on the left side models the distribution much better** by not trying too hard to model each data pattern individually.
- *In practice, overfitting causes the neural network model to perform very well during training, but the performance gets much worse during inference time when faced with brand new data.*

2. What is Regularization?

- Regularization techniques help strike a balance between fitting the training data well and maintaining the model's ability to generalize to new data.
- It involves adding a penalty term to the objective function during training, which discourages the model from learning overly complex patterns (with large weight values).
- This penalty limits the model's complexity and prevent it from fitting the noise in the training data.
- Regularization refers to a set of different techniques that lower the complexity of a neural network model during training, and thus prevent the overfitting.
- There are two popular and efficient regularization techniques called $L2$, $L1$.

L1 Regularization (Lasso Regularization)

- **Definition:** L1 regularization adds the absolute values of the weights to the loss function.
- **Mathematical Formula:**
$$\text{Loss} = \text{Original Loss} + \lambda \sum |w_i|$$
where λ is the regularization parameter and w_i are the weights.
- **Effects:**
 - Encourages sparsity in the model weights, meaning it tends to drive some weights to exactly zero.
 - This can lead to simpler models that are easier to interpret.

L2 Regularization (Ridge Regularization)

- **Definition:** L2 regularization adds the squared values of the weights to the loss function.

- **Mathematical Formula:**

$$\text{Loss} = \text{Original Loss} + \lambda \sum w_i^2$$

- **Effects:**
 - Penalizes large weights more heavily than L1, leading to weight shrinkage but not necessarily to zero.
 - Helps to stabilize the learning process by reducing the sensitivity to small fluctuations in the training data.
- **Use Case:** Useful when you want to keep all features in the model but mitigate the risk of overfitting.

What does Regularization achieve ?

- Performing L1 regularization encourages the weight values to be zero. Performing L2 regularization encourages the weight values towards zero (but not exactly zero), The effect of regularization is to **make your network learns using small weights**. Intuitively speaking smaller weights reduce the impact of some hidden neurons.
- In that case, some hidden neurons become neglectable and the overall complexity of the neural network gets reduced.
- On choosing the regularization hyperparameter λ , **the** goal is to strike the right balance between low complexity of the model and accuracy.

If λ value is too high, your model will be simple, but you run the risk of *underfitting* your data. Your model won't learn enough about the training data to make useful predictions.

If your λ value is too low, your model will be more complex, and you run the risk of *overfitting* your data. Your model will learn too much about the particularities of the training data, and won't be able to generalize to new data.

To summarize:

- ✓ Overfitting occurs in more complex neural network models (many layers, many neurons)
- ✓ Complexity of the neural network can be reduced by using L1 and L2 regularization as well as dropout
- ✓ L1 regularization forces the weight parameters to become zero
- ✓ L2 regularization forces the weight parameters towards zero (but never exactly zero)
- ✓ Smaller weight parameters make some neurons neglectable → neural network becomes less complex → less overfitting

Implementation:

```
1 keras.regularizers.l1(0.01)
2 keras.regularizers.l2(0.01)
3 keras.regularizers.l1_l2(l1=0.01, l2=0.01)
```

Weight Regularization for Dense Layers

```
1 # example of L2 on a dense layer
2 from keras.layers import Dense
3 from keras.regularizers import l2
4 ...
5 model.add(Dense(32, kernel_regularizer=l2(0.01), bias_regularizer=l2(0.01)))
6
```

<https://machinelearningmastery.com/how-to-reduce-overfitting-in-deep-learning-with-weight-regularization/>

<https://medium.com/@mjbhobe/classifying-fashion-with-a-keras-cnn-achieving-94-accuracy-part-2-a5bd7a4e7e5a>

Weight Regularization for Convolutional Layers

```
1 # example of L2 on a convolutional layer
2 from keras.layers import Conv2D
3 from keras.regularizers import l2
4 ...
5 model.add(Conv2D(32, (3,3), kernel_regularizer=l2(0.01), bias_regularizer=l2(0.01)))
6 ...
```

Examples:

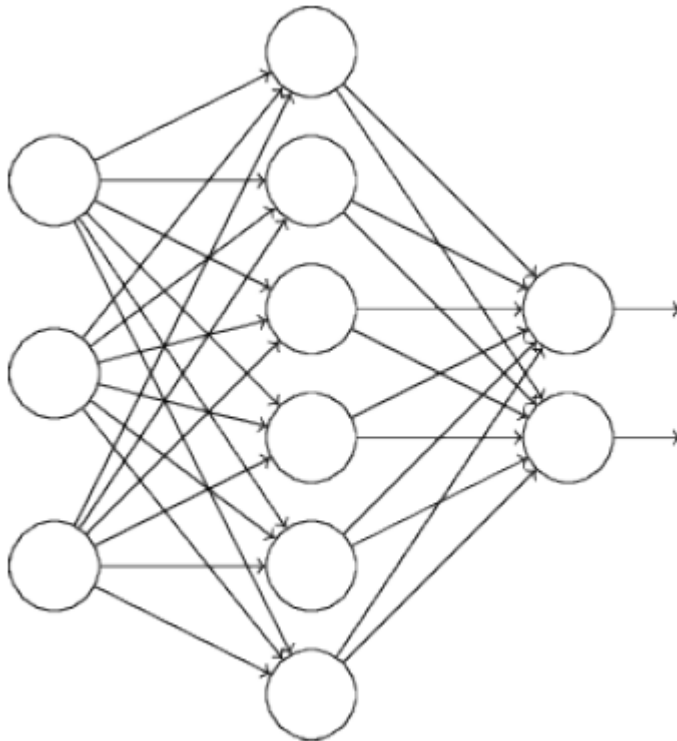
```
l2 = regularizers.l2(l2_loss_lambda)
```

```
Conv2D(64, kernel_size=(3, 3), activation='relu', padding='same', kernel_regularizer=l2,
      input_shape=(28,28,1))
```

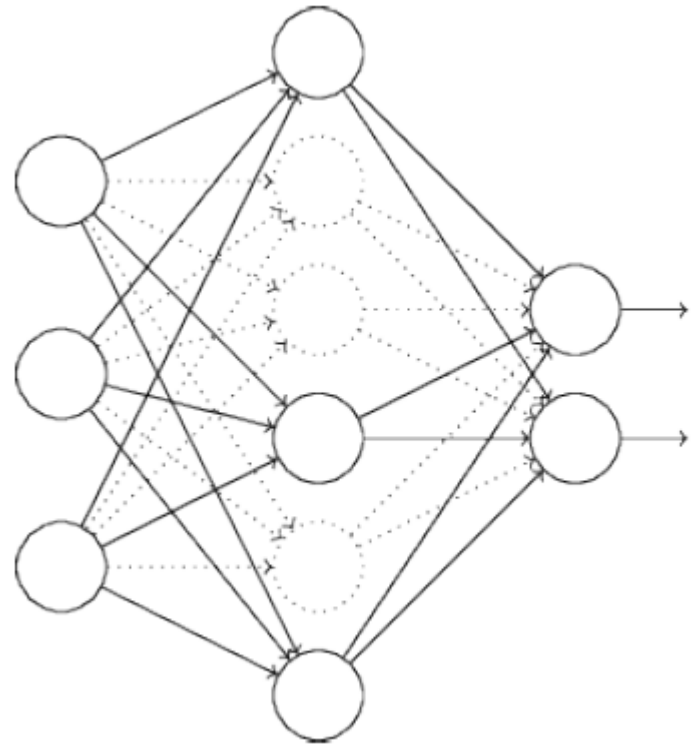
```
Dense(512, activation='relu', kernel_regularizer=l2)
```

3. Dropout Layer

Dropout does not modify the cost function. It involves randomly setting the outgoing edges of hidden units to zero at each update of the training phase.



Standard neural network



ANN with dropout of 0.5

Dropout Layer

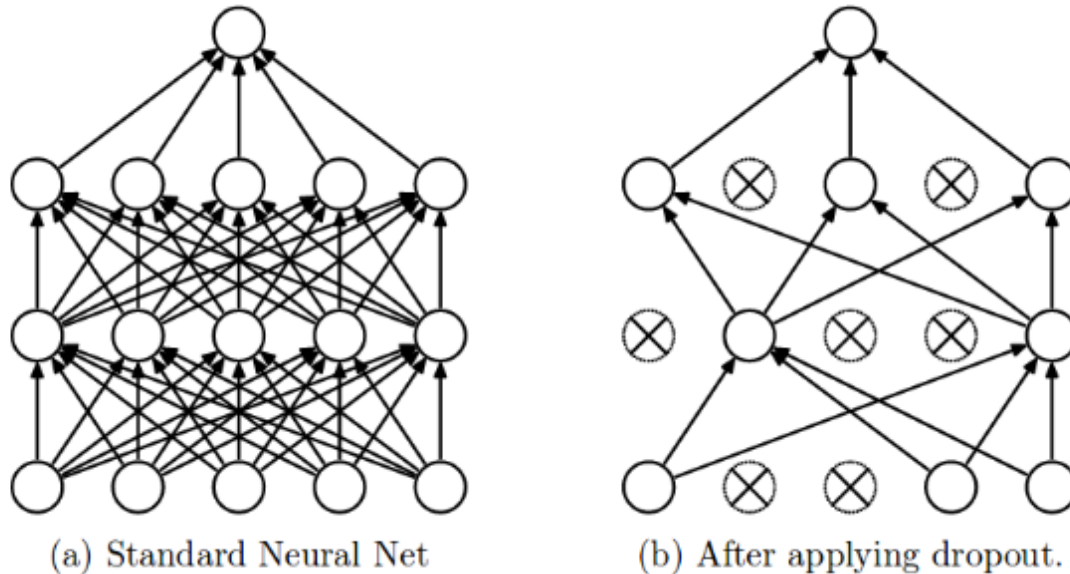


Figure : Dropout applied to a Standard Neural Network

Dropout should only be used in fully-connected layers.

It is found to be a good tool to reduce overfitting in a model. In overfitting, a unit may change in a way that fixes up the mistakes of the other units. This leads to complex co-adaptations, which would fail to generalize on the unseen dataset. Dropout breaks the inter-unit relations.

Dropout Implementation

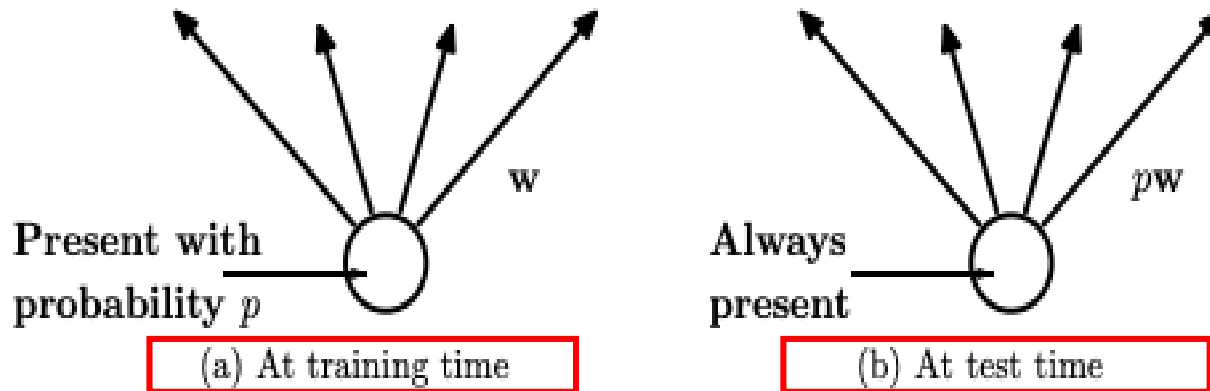


Figure : (a) A unit (neuron) during training is present with a probability p and is connected to the next layer with weights ' w ' ; (b) A unit during inference/prediction is always present and is connected to the next layer with weights, ' pw '

Dropout during inference/prediction

During the inference, we do not use a dropout layer. This means that all the units are considered during the prediction step.

But, because of taking all the units/neurons from a layer, the final weights will be larger than expected and to deal with this problem, weights are first scaled by the chosen dropout rate. With this, the network would be able to make accurate predictions.

To be more precise, if a unit is retained with probability p during training, the outgoing weights of that unit are multiplied by p during the prediction stage.

Ref:

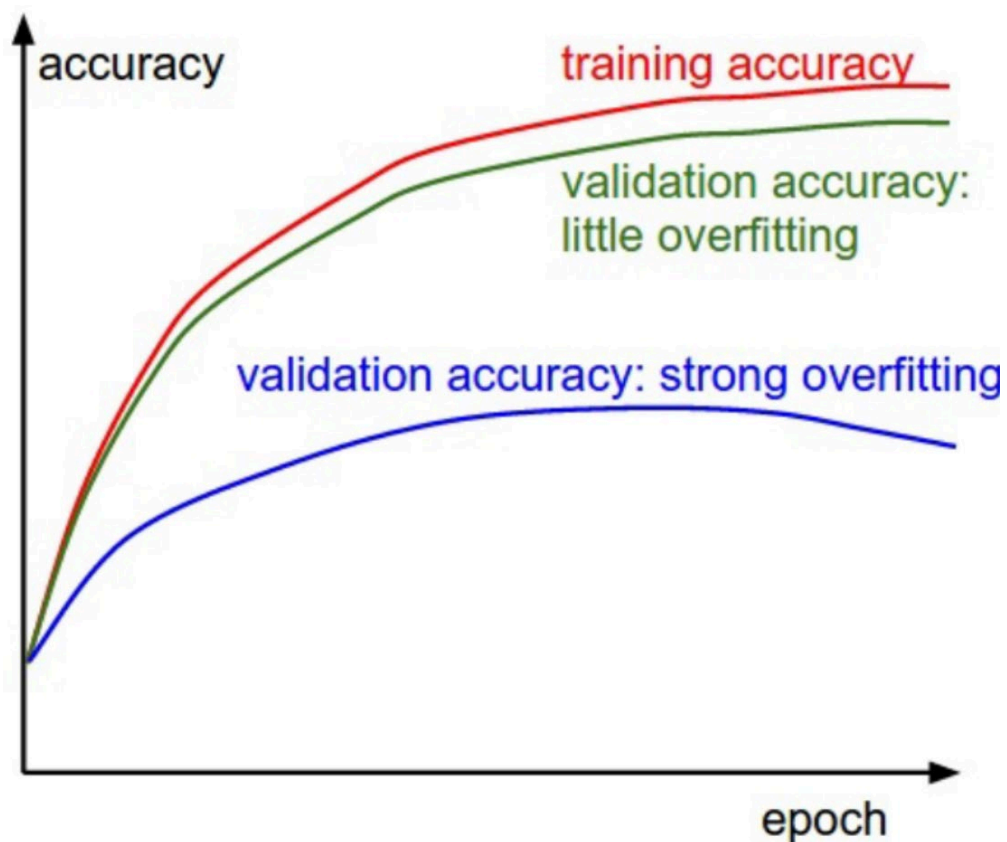
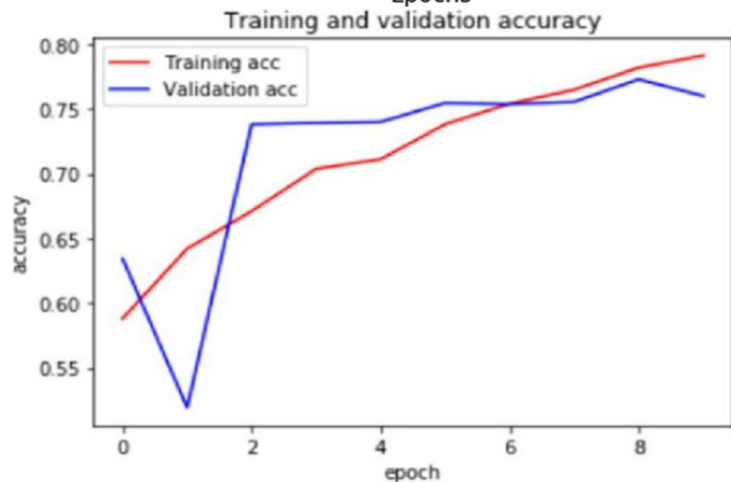
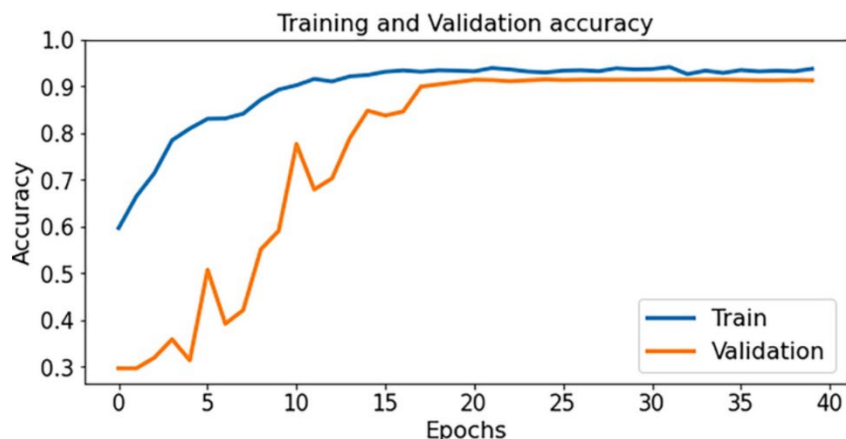
<https://towardsdatascience.com/dropout-in-neural-networks-47a162d621d9>

- During dropout, some neurons get deactivated with a random probability $p \rightarrow$ Neural network becomes less complex $\rightarrow$ less overfitting

Dropout has been found useful in training deep networks with problem of overfitting.

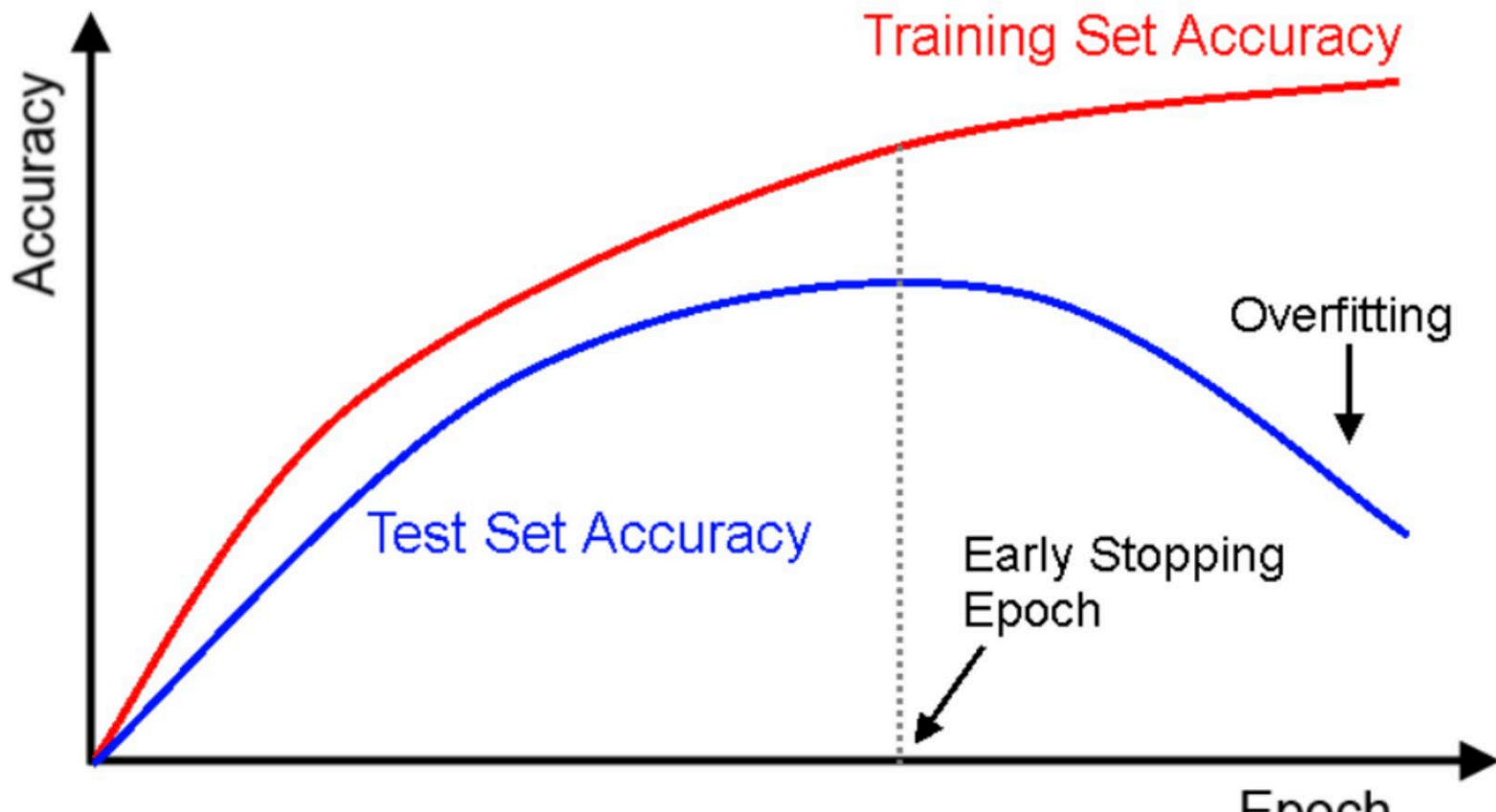
4. Early Stopping

- Early stopping is a technique where the training is stopped before the model fully converges to the training data.
- It involves monitoring the model's performance on a validation set during training and stopping the training when the performance on the validation set starts to degrade.



Early Stopping

- Early stopping prevents the model from overfitting to the training data and helps find the optimal point of generalization.



5. Batch Normalization

Batch Normalization (Batch Norm) is a widely used technique in the field of Deep Learning **to improve the learning speed** of Neural Networks.

Normalization is a pre-processing technique used to standardize data. In a typical situation, the input may have very different sources of data which has different range. **If the data is not normalized, it may cause problem when training our network,** make it drastically harder to train and decrease its learning speed.

An example: to determine the price/value of a used car based on two features : the **age** in years and the **total amount of kilometers** it has been driven for.

These can have very different ranges, ranging from 0 to 30 years, while distance could go from zero up to hundreds of thousands of kilometers. We don't want features to have these differences in ranges, as the value with the higher range might bias our models into giving them inflated importance.

Introduction to BN

There are two main methods **to normalize our input data**.

Method A: Normalization of the input data, to scale it to a range from 0 to 1 :

$$x_{normalized} = \frac{x - m}{x_{max} - x_{min}}$$

x the data point to normalize, m the mean of the data set, x_{max} the highest value, and x_{min} the lowest value. The non-normalized input data points with wide ranges can cause instability in Neural Networks. The relatively large inputs can cascade down to the layers, causing problems such as exploding gradients.

Method B: The other technique used to normalize data is forcing the data points to have a mean of 0 and a standard deviation of 1, using the following formula:

$$x_{normalized} = \frac{x - m}{s}$$

x is the data point to normalize, m the mean of the data set, and s the standard deviation of the data set. Now, each data point mimics a standard normal distribution. Having all the features on this scale, none of them will have a bias, and therefore, our models will learn better.

Batch Normalization

Batch Norm is a normalization technique done between the layers of a neural network instead of the raw/input data.

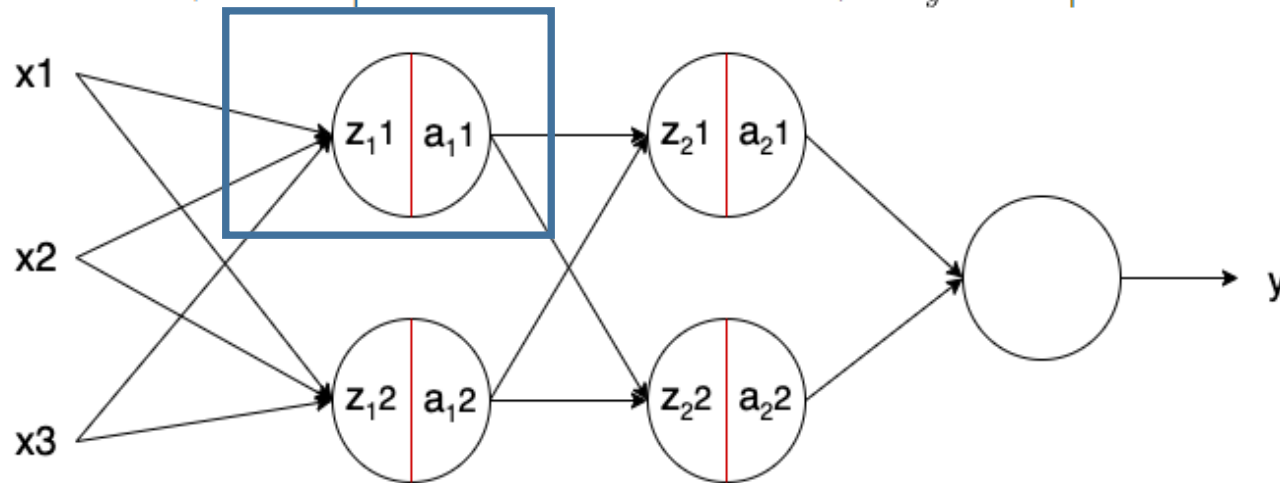
It serves to speed up training and use higher learning rates, making learning easier.

We can define the normalization formula of Batch Norm as:

$$z^N = \left(\frac{z - m_z}{s_z} \right)$$

m_z is the mean of the neurons' output and s_z the standard deviation of the neurons' output.

In the following image, we can see a regular feed-forward Neural Network: x_i are the inputs, z the output of the neurons, a the output of the activation functions, and y the output of the network:



Batch Norm – in the image represented with a red line – is applied to the neurons' output just before applying the activation function. Usually, a neuron without Batch Norm would be computed as follows:

$$z = g(w, x) + b; \quad a = f(z)$$

being $g()$ the linear transformation of the neuron, w the weights of the neuron, b the bias of the neurons, and $f()$ the activation function. The model learns the parameters w and b . Adding Batch Norm, it looks as:

$$z = g(w, x);$$

$$z^N = \left(\frac{z - m_z}{s_z} \right) \cdot \gamma + \beta; \quad a = f(z^N)$$

**BN before
activation function f**

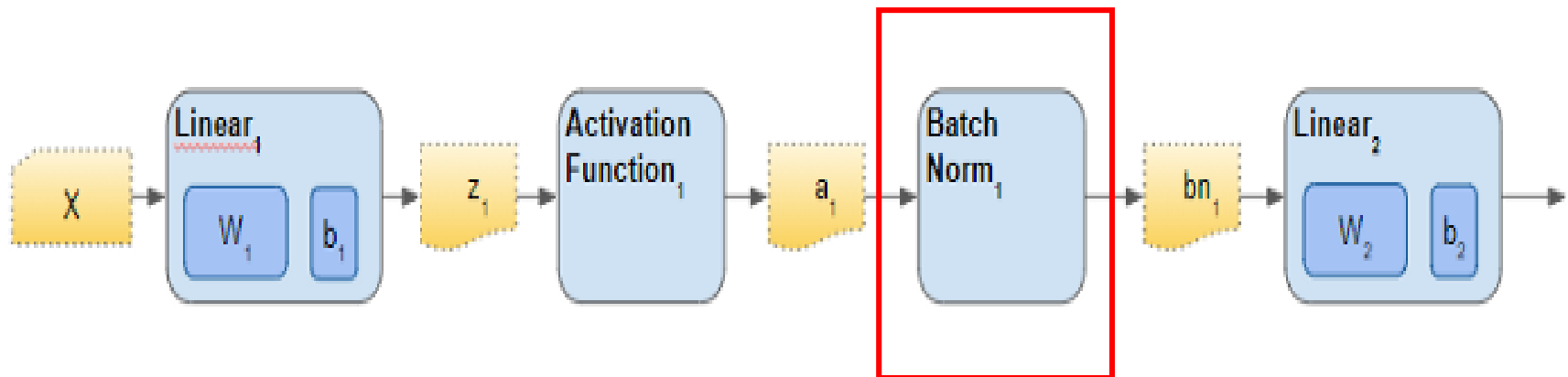
being z^N the output of Batch Norm, m_z the mean of the neurons' output, s_z the standard deviation of the output of the neurons, and γ and β learning parameters of Batch Norm. Note that the bias of the neurons (b) is removed. This is because as we subtract the mean m_z , any constant over the values of z – such as b – can be ignored as it will be subtracted by itself.

learning parameters

The parameters *beta* and *gamma* shift the mean and standard deviation, respectively. These values are learned over epochs and the other learning parameters, such as the weights of the neurons, aiming to decrease the loss of the model.

How Does Batch Norm work?

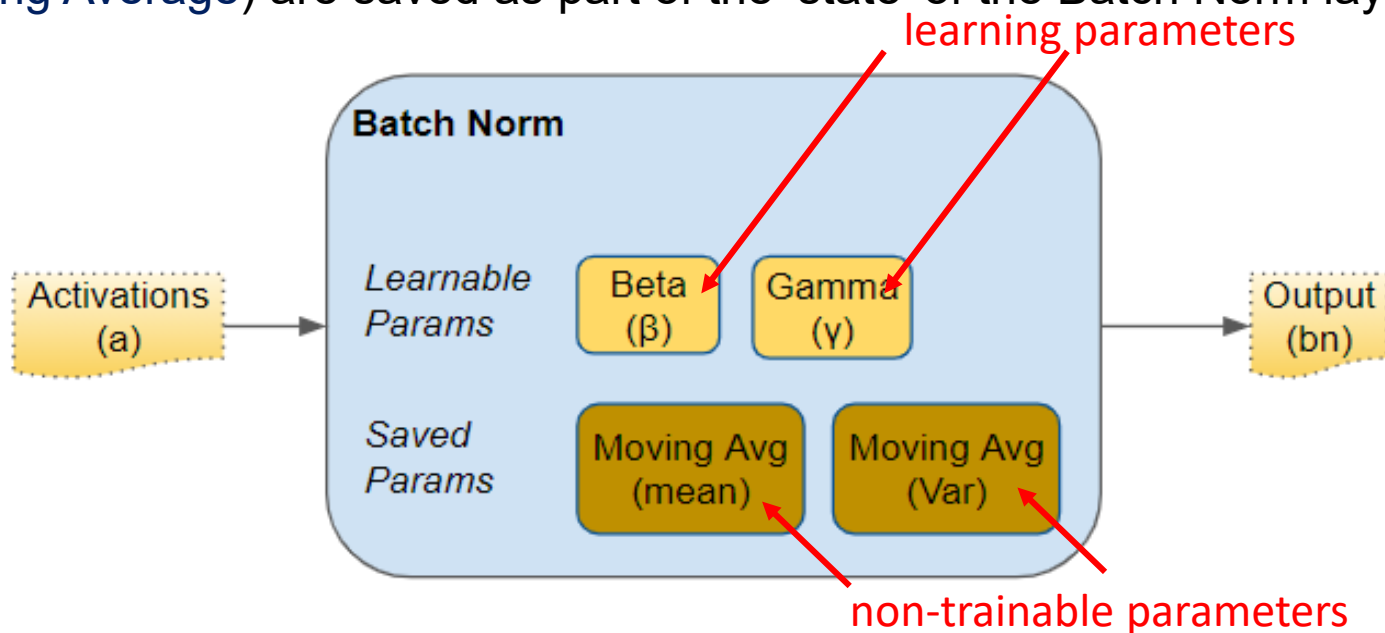
Batch Norm is just another network layer that gets inserted between a hidden layer and the next hidden layer. Its job is to **take the outputs from the first hidden layer and normalize them before passing them on as the input of the next hidden layer.**



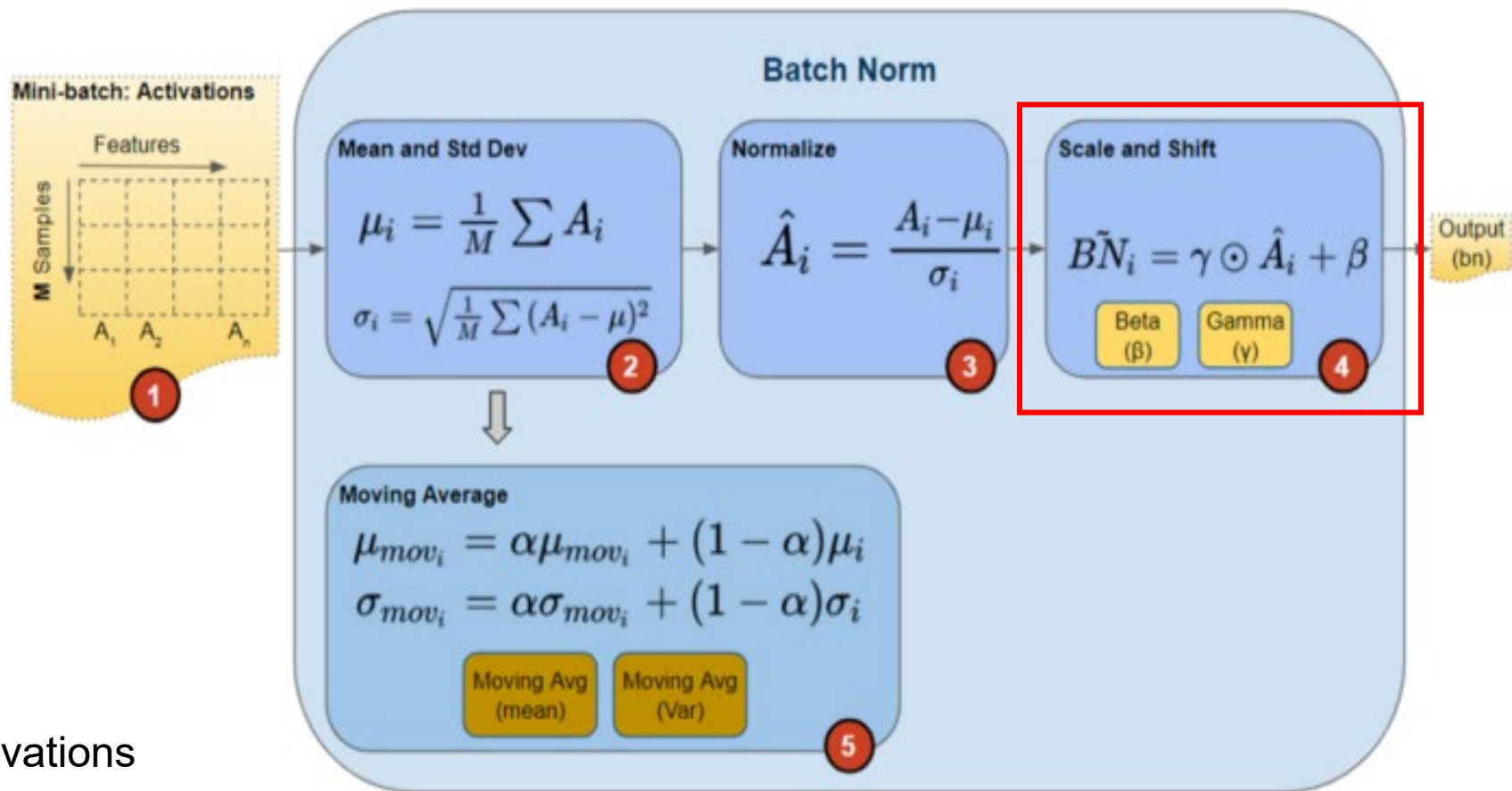
BN after (or before)
activation function f

Just like the parameters (eg. weights, bias) of any network layer, a Batch Norm layer also has parameters of its own:

- Two learnable parameters called **beta** and **gamma**.
- Two non-learnable parameters (**Mean Moving Average** and **Variance Moving Average**) are saved as part of the 'state' of the Batch Norm layer.



During training, we feed the network one **mini-batch of data** at a time. During the forward pass, each layer of the network processes that mini-batch of data. The Batch Norm layer processes its data as follows:



1. Activations

The activations from the previous layer are passed as input to the Batch Norm. There is one activation vector for each feature in the data. That is we have one batch of input data of each feature.

2. Calculate Mean and Variance

For each activation vector separately, calculate the mean and variance of all the values in the mini-batch.

3. Normalize

Calculate the normalized values for each activation feature vector using the corresponding **mean and variance**. These normalized values now have zero mean and unit variance.

4. Scale and Shift

Batch Norm allows its values to be shifted (to a different mean) and scaled (to a different variance). It does this by multiplying the normalized values by a factor, **gamma**, and adding to it a factor, **beta**.

What makes this innovation ingenious is that these factors are not hyperparameters (ie. constants provided by the model designer) but are trainable parameters that are learned by the network. In other words, each Batch Norm layer is able to optimally find the best factors for itself, and can thus shift and scale the normalized values to get the best predictions.

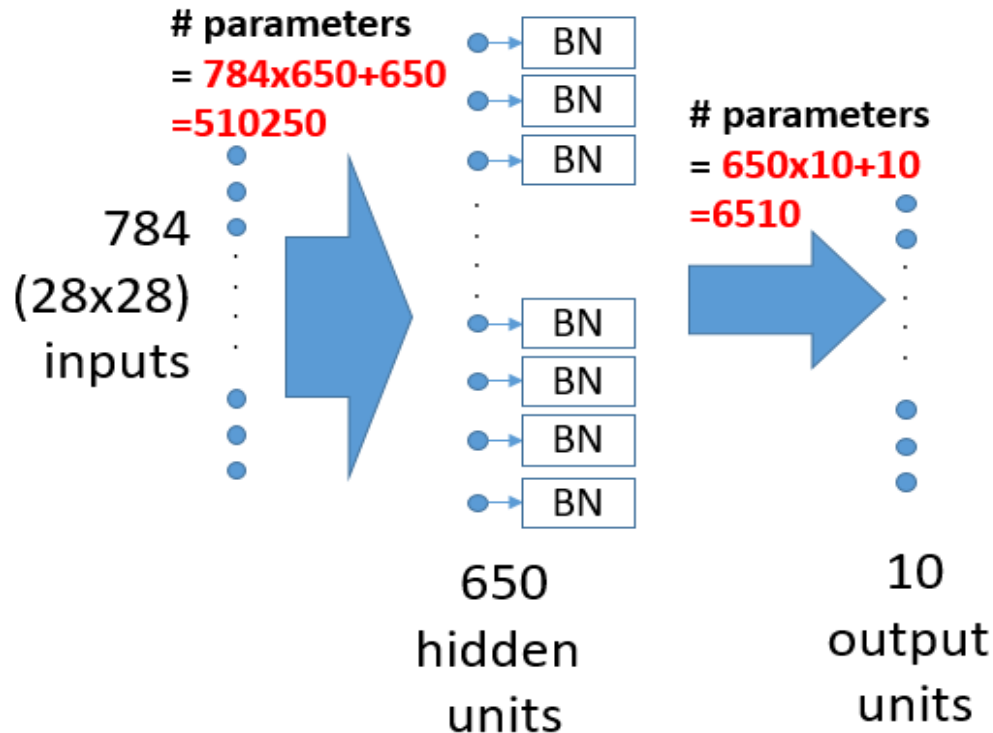
5. Moving Average

In addition, Batch Norm also keeps a running count of the Exponential Moving Average (EMA) of the mean and variance. During training, it simply calculates this EMA but does not do anything with it. At the end of training, it simply saves this value as part of the layer's state, for use during the Inference phase.

Batch Normalization in Artificial Neural Networks (ANN)

The example with MNIST, 784 inputs (28x28)

Example A: MNIST using ANN



#BN parameters = 650×4 = **2600**

```
X_train.shape after reshape (60000, 784)
num_pixels = 784
num_hidden_nodes = 650
```

Class05d-BN_MNIST.ipynb

```
# EXAMPLE A2
```

```
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(num_hidden_nodes, input dim=num_pixels, kernel_initializer='normal', activation='relu'))
    model.add(BatchNormalization())
    model.add(Dense(num_classes, kernel_initializer='normal', activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    # Choice of optimizer: adam (adaptive moment estimation), AdaGrad (adaptive learning rate),
    # sgd (Stochastic gradient descent), RMSprop (similar to AdaGrad), Adadelta (adaptive delta) ...
    return model

# build the model
model = baseline_model()
model.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 650)	510250
batch_normalization (Batch Normalization)	(None, 650)	2600
dense_3 (Dense)	(None, 10)	6510

```
=====  
Total params: 519,360
```

```
Trainable params: 518,060
```

```
Non-trainable params: 1,300
```

$(784 \times 650 + 650)$

Notice this addition

$518,060 + 1300$

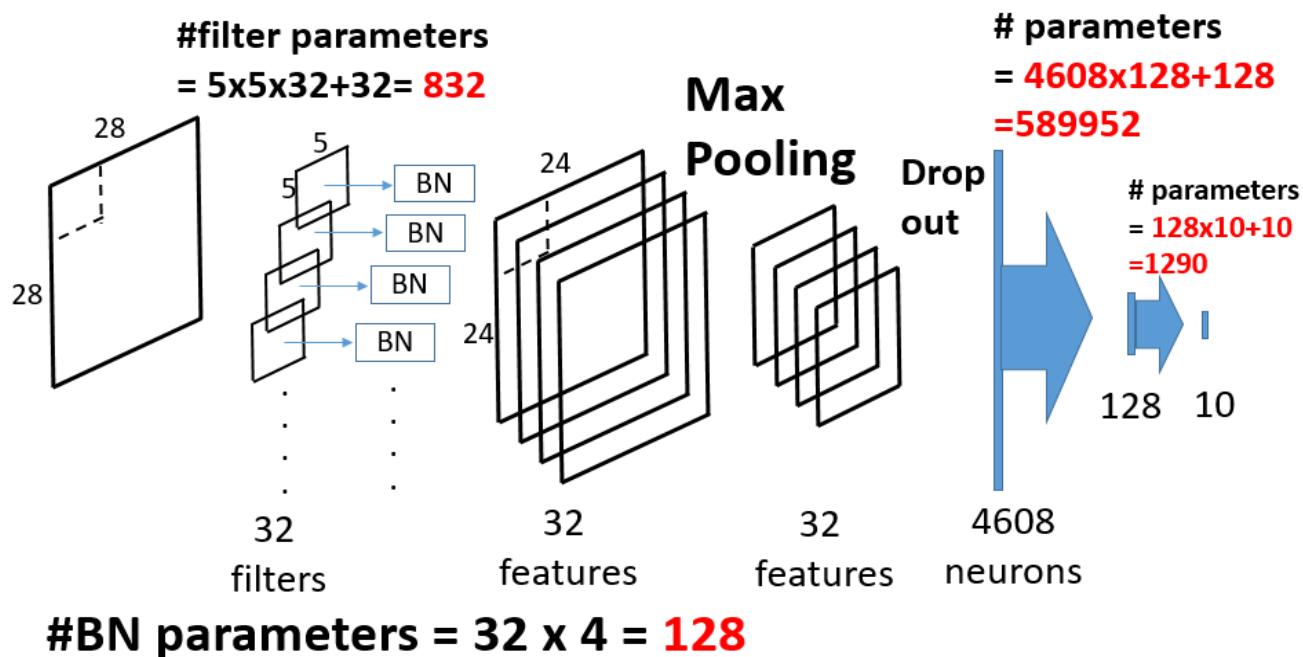
$510250 + 1300 + 6510$

Batch Normalization in Convolutional Neural Networks

Batch Norm works in a very similar way in Convolutional Neural Networks. In convolutions, we have shared filters that go along the feature maps of the input. These filters are the same on every feature map. It is then reasonable to normalize the output, in the same way, sharing it over the feature maps.

In other words, this means that the parameters used to normalize are calculated along with each entire feature map. In a regular Batch Norm, each feature would have a different mean and standard deviation. Here, **each feature map will have a single mean and standard deviation.**

Example B: MNIST using CNN



Class05d-BN_MNIST.ipynb

```
[7] # EXAMPLE B2
# define baseline model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Conv2D(32, (5, 5), input_shape=(28, 28, 1), activation='relu'))
    model.add(BatchNormalization())
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

model = baseline_model()
model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #	
conv2d_1 (Conv2D)	(None, 24, 24, 32)	832	$5 \times 5 \times 32 + 32 = 832$
batch_normalization_1 (Batch Normalization)	(None, 24, 24, 32)	128	$32 \times 4 = 128$
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 32)	0	
dropout_1 (Dropout)	(None, 12, 12, 32)	0	
flatten_1 (Flatten)	(None, 4608)	0	
dense_6 (Dense)	(None, 128)	589952	$4608 \times 128 + 128 = 589952$
dense_7 (Dense)	(None, 10)	1290	$128 \times 10 + 10 = 1290$
=====			
Total params: 592,202			$592,138 + 64$
Trainable params: 592,138			$832 + 64 + 589952 + 1290$
Non-trainable params: 64			

Demo File: Class05d-BN_MNIST.ipynb

```
[9] # EXAMPLE C2
# define the larger model
def larger_model():
    # create model
    model = Sequential()
    model.add(Conv2D(30, (5, 5), input_shape=(28, 28, 1), activation='relu'))
    model.add(BatchNormalization())
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Conv2D(15, (3, 3), activation='relu'))
    model.add(BatchNormalization())
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(BatchNormalization())
    model.add(Dense(50, activation='relu'))
    model.add(BatchNormalization())
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# build the model
model = larger_model()
model.summary()
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 24, 24, 30)	780
batch_normalization_2 (Batch Normalization)	(None, 24, 24, 30)	120
max_pooling2d_4 (MaxPooling2D)	(None, 12, 12, 30)	0
conv2d_5 (Conv2D)	(None, 10, 10, 15)	4065
batch_normalization_3 (Batch Normalization)	(None, 10, 10, 15)	60
max_pooling2d_5 (MaxPooling2D)	(None, 5, 5, 15)	0
dropout_3 (Dropout)	(None, 5, 5, 15)	0
flatten_3 (Flatten)	(None, 375)	0
dense_11 (Dense)	(None, 128)	48128
batch_normalization_4 (Batch Normalization)	(None, 128)	512
dense_12 (Dense)	(None, 50)	6450
batch_normalization_5 (Batch Normalization)	(None, 50)	200
dense_13 (Dense)	(None, 10)	510

30x4
= 120

15x4
= 60

128x4
= 512

50x4
= 200

=====
Total params: 60,825

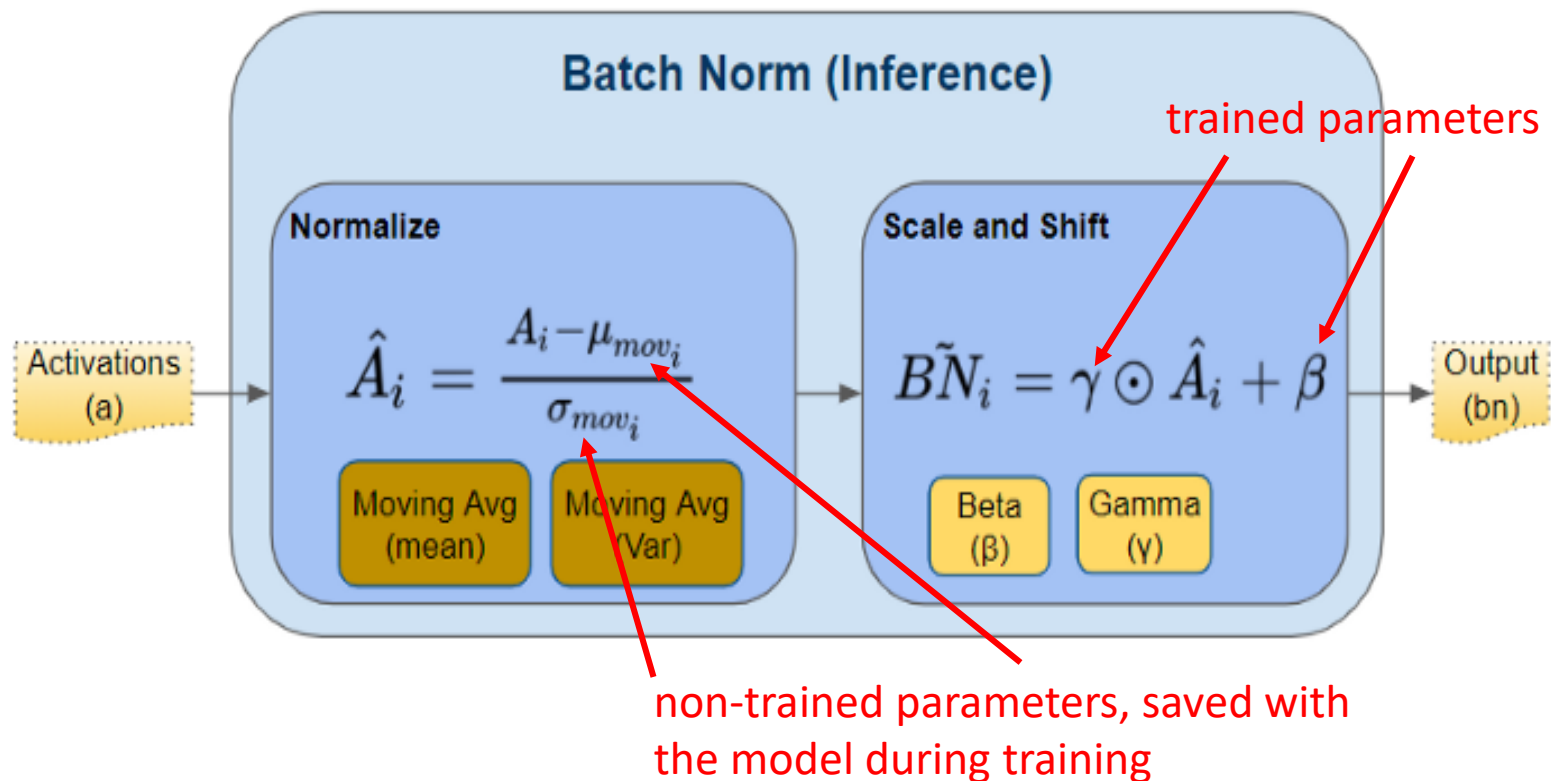
Trainable params: 60,379

Non-trainable params: 446 ← (120+60+512+200)/2 = 446 (half are non-trainable) ⁴²

Batch Norm during Inference

As we discussed above, during Training, Batch Norm starts by calculating the mean and variance for a mini-batch. However, during Inference, we have a single sample, not a mini-batch. How do we obtain the mean and variance in that case?

Here is where **the two Moving Average parameters** come in — the ones that we calculated during training and **saved** with the model. We use those **saved mean and variance values for the Batch Norm during Inference**.



6. Data Augmentation

Data augmentation is a technique used to artificially **increase the size of a training dataset** by applying various transformations to the existing data. This technique is commonly used in machine learning and deep learning tasks, especially in computer vision, to improve the generalization and robustness of the trained models.

Why Use Data Augmentation?

- **Increased Dataset Size:** By creating new samples from the existing data, data augmentation effectively increases the size of the dataset, which can lead to better model performance.
- **Regularization:** **Data augmentation introduces additional variations in the data, which can help prevent overfitting by providing the model with a more diverse set of examples.**
- **Improved Generalization:** By exposing the model to a wider range of variations in the data, data augmentation helps the model generalize better to unseen examples.

Common Data Augmentation Techniques

- **Rotation:** Rotate the image by a certain angle (e.g., 90 degrees, 180 degrees).
- **Translation:** Shift the image horizontally or vertically by a certain distance.
- **Scaling:** Enlarge or shrink the image by a certain factor.
- **Flipping:** Flip the image horizontally or vertically.
- **Shearing:** Skew the image along the x or y-axis.
- **Zooming:** Zoom in or out of the image.
- **Brightness Adjustment:** Increase or decrease the brightness of the image.
- **Contrast Adjustment:** Increase or decrease the contrast of the image.
- **Noise Addition:** Add random noise to the image.

Original

Rotation

Blur

Contrast

Scaling

Illumination

Projective



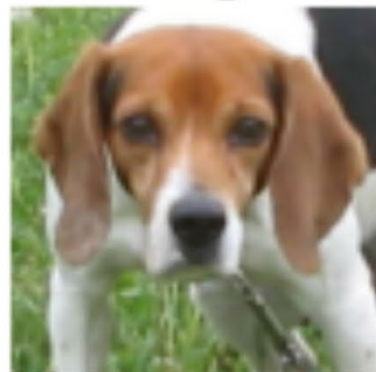
dog



dog



dog



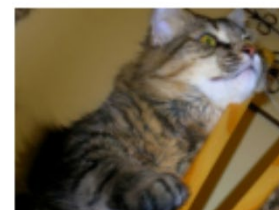
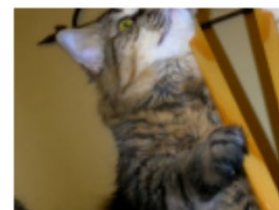
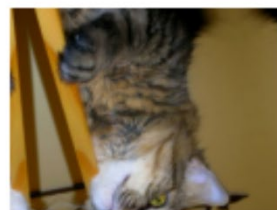
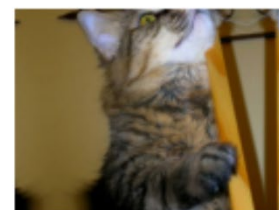
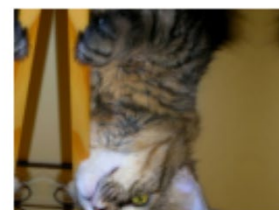
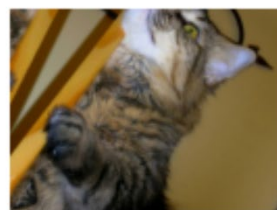
cat



Original



Image Shearing



Data Augmentation

- Particularly useful in tasks involving **images and audio**, data augmentation **artificially increases the size and diversity of the training dataset by applying random, but realistic, transformations to the training data.**
- For **images**: rotating, scaling, and cropping of images.
For **audio**: altering the pitch and speed of audio clips.
- This method helps the model learn more robust features by exposing it to **varied forms of its input data**, thereby improving its ability to generalize well to new, unseen examples.
- It can be seen as a form of training-time augmentation that enhances model generalization without changing the underlying model architecture.

Example of how to use data augmentation with Keras

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
# Define the data augmentation parameters
```

```
datagen = ImageDataGenerator(  
    rotation_range=20,  
    width_shift_range=0.1,  
    height_shift_range=0.1,  
    shear_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True,  
    vertical_flip=True,  
    fill_mode='nearest'  
)
```

```
# Load the original image
```

```
img = load_image('path/to/image.jpg')
```

```
# Reshape the image to a 4D tensor with shape (1, height, width, channels)
```

```
img = img.reshape((1,) + img.shape)
```

```
# Generate augmented images
augmented_images = []

for batch in datagen.flow(img, batch_size=1):
    augmented_images.append(batch[0])
    if len(augmented_images) >= 5:
        break # Stop the loop after generating 5 augmented images

# Display the original and augmented images
for i, img in enumerate(augmented_images):
    plt.subplot(1, 5, i+1)
    plt.imshow(img)
    plt.axis('off')
plt.show()
```

Datagen provides a powerful platform that allows one to generate domain-specific, simulated synthetic data..

7. Further comments and Conclusions

7.1 The **hyperparameter** in L1 and L2 regularization controls the weight of the penalty term in the objective function. Higher values lead to stronger regularization and simpler models, while lower values allow for more complex models. The optimal value depends on the specific problem, data characteristics, and the desired trade-off between bias and variance.

7.2 **Dropout probability** in dropout regularization, which determines the fraction of input units to be randomly set to zero during training.

7.3 **Cross-validation** is often used in combination with hyperparameter tuning to estimate the model's generalization performance and avoid overfitting to the validation set. **Cross-validation involves splitting the data into multiple subsets, training the model on a subset, and evaluating it on the remaining subsets.** This process is repeated multiple times, and the average performance across the folds is used as an estimate of the model's generalization performance.

7.4 **Proper tuning of regularization hyperparameters** is essential for building accurate and robust machine learning models.

Conclusions

- The crucial role of regularization in machine learning **to combat the common problem of overfitting.**
- Through techniques like L1 and L2 regularization, dropout, batch normalization, and data augmentation, they help models generalize better to new data, thus enhancing their real-world applicability.
- Implementing these techniques properly can significantly improve a model's performance, **making it robust against variations in unseen data.**
- Ultimately, regularization in machine learning not only refines the model's capacity to learn but also ensures it can perform reliably in various operational environments.